

# CSC311 Machine Learning Challenge - Final Report

Rhea Paste, Shreya Sirgound

## 1. Data

### 1.1 Overview and cleaning

The dataset contains 1,462 labeled survey responses across four target cities (Dubai, New York City, Paris, and Rio de Janeiro), with roughly balanced class counts (367 / 365 / 365 / 365). Ten questions (Q1–Q10) were available as features; we used Q1–Q7 and Q10, and deliberately excluded Q8 and Q9.

### 1.2 Why Q8 and Q9 were dropped

Q8 ("how many different languages might you overhear during a stroll") and Q9 ("how many different fashion styles might you spot in a 10-minute walk") ask respondents to produce open-ended numeric estimates with no natural upper bound. In the raw data, Q8 has a mean of 4.8 but a maximum of 800, which is an implausible answer for a walk through any city, and a standard deviation of 22.1, more than four times the mean. This indicates the question invites wildly inconsistent interpretations, so any signal is swamped by noise. Q9 has the same open-ended structure and is similarly prone to arbitrary guesses. Additionally, our decision tree model for Q8 produced a final validation accuracy of 39.41%, and our decision tree model for Q9 produced a final validation accuracy of 34.15%. Both these validation accuracies are incredibly weak in comparison to the other submodels, and coupled with the already poor data correlation, it was clear a Q8 and Q9 model would only poorly impact the final model. Rather than trying to salvage these with aggressive clipping (which would itself set an arbitrary threshold with no objective justification), we chose to exclude both questions and rely on the eight remaining questions.

### 1.3 Exploring each feature and its relationship to the target

Q1–Q4 are 1–5 ratings (popularity, social media virality, architectural uniqueness, and street-party enthusiasm). Boxplots of each rating by city (Figure 1) show real, if noisy, separation: New York City and Paris are rated most consistently "popular" (Q1); New York City is most likely to go "viral" (Q2); Rio de Janeiro stands out for street-party enthusiasm (Q4); Dubai and Paris score highest on architectural uniqueness (Q3). None of the four ratings cleanly separates all four cities alone, but each contributes a different axis of separation, which gave us the signal that an ensemble model should be designed to combine these results.

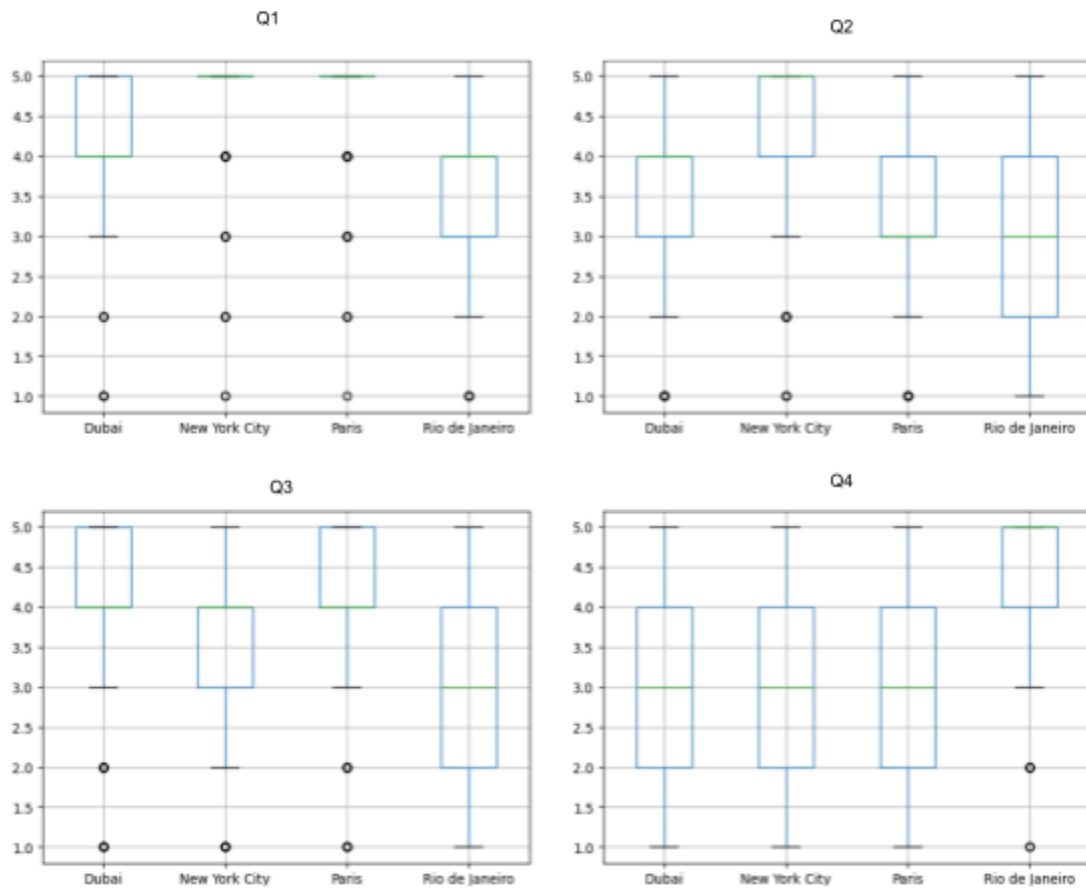


Figure 1: Q1–Q4 rating boxplot distributions by city.

Q6 asks respondents to rank six words/categories (Skyscrapers, Sport, Art and Music, Carnival, Cuisine, and Economic) from 1 (least relatable) to 6 (most relatable) for the city they were assigned. This turned out to be our strongest single-question signal. Figure 2 shows the mean rank of each category per city: Rio de Janeiro is strongly associated with Carnival, Dubai and NYC with Economic and Skyscrapers, and Paris with Art and Music. These patterns all seem to line up with cultural stereotypes that each question was clearly designed around.

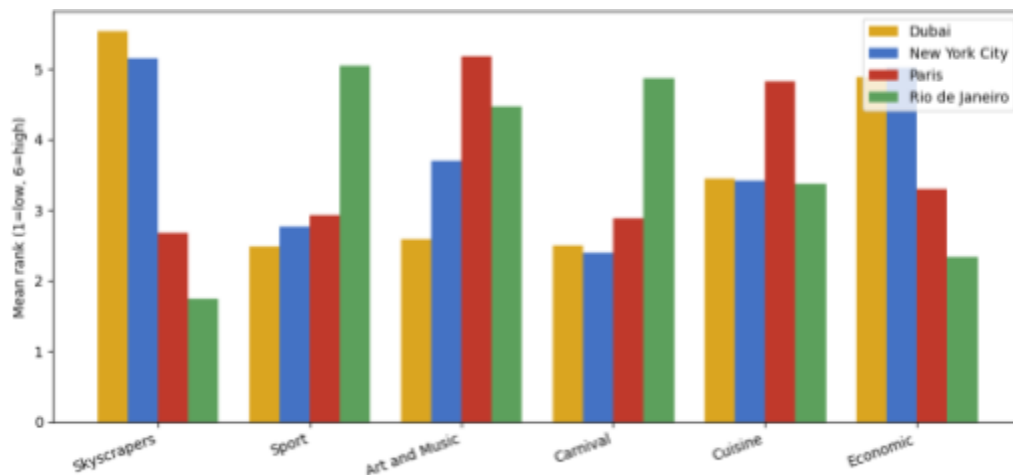


Figure 2: Q6 mean self-reported category ranking by city.

Q7 asks respondents to guess the city's average January temperature in Celsius. We found that Rio de Janeiro and Dubai cluster around 22–25°C, while New York City and Paris cluster around 3–7°C (Figure 3). The value is almost perfectly bimodal along the hot/cold axis, but far weaker at distinguishing Dubai from Rio or New York from Paris within each pair, further signaling that this is the kind of question that needs to be combined with others to fully resolve the label. Three responses fell far outside any plausible January average (89°C, 10,000°C, and 1,000°C) and led us to clip the data to a plausible  $[-30, 45]^{\circ}\text{C}$  range before use.

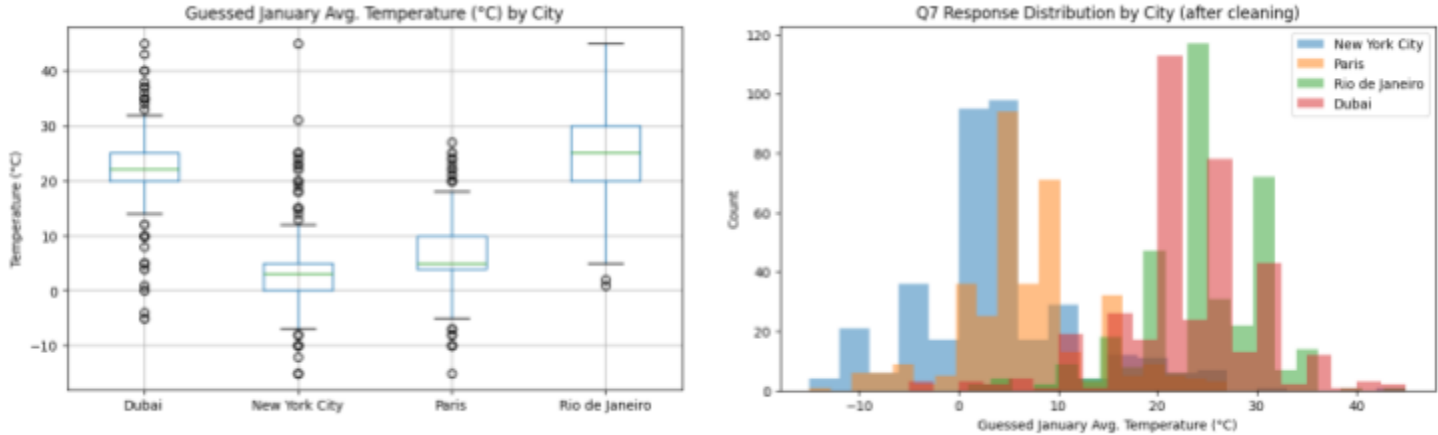


Figure 3: Boxplot and overlaid histograms for Q7.

Q5 (who you'd travel to the city with: Co-worker / Friends / Partner / Siblings, multi-select) and Q10 (a free-text quote associated with the city) were also explored. Q5 turned out to be our weakest feature since city choice doesn't obviously determine travel companions, and this is confirmed empirically in Section 3. Q10 required keyword-based feature engineering: we manually inspected the most frequent words associated with each city (e.g. "burj", "khalifa", "desert" for Dubai; "eiffel", "baguette", "romance" for Paris; etc.) and built 39 binary indicator features, one per keyword, based on whether each word appears anywhere in the response.

## 1.4 Feature representation

| Question | Representation                                                                                                                            | Dimensionality |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| Q1–Q4    | Degree-2 polynomial expansion (linear + squared + pairwise cross terms) of the four 1–5 ratings, standardized using training-set mean/std | 14 + bias      |
| Q5       | Multi-hot indicator over 5 fixed categories (Co-worker, Friends, Partner, Siblings, missing→"Unknown")                                    | 5 + bias       |
| Q6       | Six numeric ranks (1–6), standardized using training-set mean/std                                                                         | 6 + bias       |
| Q7       | Cleaned, clipped to $[-30, 45]^{\circ}\text{C}$ , standardized single continuous feature                                                  | 1 + bias       |
| Q10      | Binary indicator per keyword (39 hand-selected keywords, 9–10 per city), giving a bag-of-keywords, not full bag-of-words                  | 39             |

Missing values (present only in Q2, Q3, Q7, and occasional blank Q5/Q6/Q10 entries) were imputed using statistics computed from the training split only, never from validation or test data to avoid data leakage. Q1–Q4 and Q6 were standardized (zero mean, unit variance using training statistics) because their raw scales differ dramatically (linear terms span 1–5, squared terms span 1–25); without

standardization, gradient descent converges far more slowly and to a worse optimum within a fixed epoch limit, which we confirmed empirically while debugging our first draft models (see Section 3).

## 1.5 Data splitting

We use a three-way split, computed once and reused identically across every sub-model so that all consequent comparisons are on the same rows:

- `global_train_idx` (70%, 1,022 rows): used to fit every sub-model's final parameters.
- `train_idx` / `val_idx` (a further 80/20 split of the 70%, i.e. 817 / 205 rows): used only for sub-model training and hyperparameter search, never for final parameter fitting.
- `global_val_idx` (10%, 147 rows): held out from every sub-model's training and tuning; used only to decide how to combine sub-model outputs (the late-fusion step).
- `global_test_idx` (20%, 293 rows): touched by nothing above; used exactly once for the final accuracy estimate reported in Section 4.

All splits are stratified by label (via scikit-learn's `train_test_split`, used only for exploration/splitting) so each split preserves the roughly 25/25/25/25 class balance.

## 2. Model Exploration

We explored four model families in total, satisfying the requirement to consider 3+ families.

### 2.1 Softmax (multinomial) logistic regression

This is our primary family, used for four of the five sub-models (Q1–4, Q5, Q6, Q7). We implemented multinomial logistic regression by hand: a linear layer producing 4 logits, a numerically stable softmax, categorical cross-entropy loss, and full-batch or stochastic gradient descent (with optional learning-rate decay for SGD). Each of the four sub-models is trained independently on its own feature block, producing a 4-dimensional class-probability vector per row.

### 2.2 Decision tree

Used for the Q10 sub-model, whose 39-dimensional binary keyword-indicator features are a natural fit for axis-aligned splits. We fit the tree with scikit-learn during exploration (entropy criterion, tuned `max_depth`), then re-implemented the fitted tree's logic in pure NumPy.

### 2.3 K-Nearest Neighbors (explored, not used in final model)

As a third family, we tried KNN on the full concatenated ("early fusion") feature matrix (all questions combined into one 65-dimensional vector per row, Q10 as raw indicators, everything else standardized). We tested with `k` from 3 to 51 on the `train_idx`→`val_idx` split; `k=9` was best (86.8% validation accuracy, 86.4% test accuracy). This is a strong result for a single model, but it requires storing and scanning the full training set at prediction time and has no interpretable per-question breakdown, which made it a worse fit for our late-fusion design.

### 2.4 Gaussian Naive Bayes (explored, not used in final model)

As a fourth family, we tried Gaussian Naive Bayes on the same combined feature matrix. It performed significantly worse (60.8% test accuracy). This was expected since several of our features are binary/categorical rather than Gaussian-distributed, which violates Naive Bayes' core assumption, and

Naive Bayes cannot model the interaction terms (e.g.,  $Q1 \times Q2$ ) that our polynomial expansion for  $Q1-4$  relies on.

## 2.5 Early fusion vs. late fusion

Beyond the four model families, we also directly compared two ensembling philosophies: "early fusion" (concatenate all raw features into one vector, train a single classifier) vs. "late fusion" (train one classifier per question, then combine their output probabilities). We tested an early-fusion logistic regression on the combined 65-dimensional feature vector using sklearn and compared it against our per-question late-fusion ensemble.

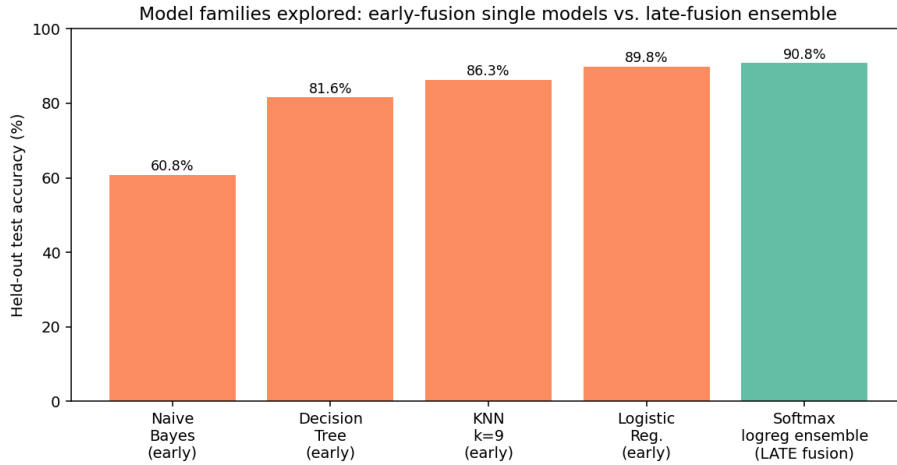


Figure 4: Test accuracy of every model family / fusion strategy explored, on the same held-out `global_test_idx` split (`seed=42`).

Late fusion of five independently-tuned sub-models outperformed every early-fusion alternative we tried. We believe this is because late fusion lets each sub-model specialize on the scale and structure of its own question (e.g.,  $Q7$  needs standardization and no interaction terms;  $Q1-4$  benefit from polynomial interactions;  $Q10$  is naturally categorical) rather than forcing one model to find a single decision boundary across 65 heterogeneous dimensions at once. Late fusion also made per-question analysis and debugging (Section 3) much easier.

## 3. Model Choice and Hyperparameters

### 3.1 Ensuring comparable evaluation

Every sub-model, and the meta-fusion layer, is evaluated on the same fixed, stratified index splits described in Section 1.4, computed once with a fixed random seed (`seed=42` used in submission) and reused everywhere. Sub-model hyperparameters (learning rate, batch vs. SGD, decision-tree depth) are selected purely on `train_idx`→`val_idx`, and are never touched again once a final choice is made. The sub-models are then refitted on the full `global_train_idx` (70%) pool using that fixed hyperparameter choice. The late-fusion combination strategy is chosen using only `global_val_idx` (147 rows untouched by any sub-model), split further into an internal 70/30 tune check. `global_test_idx` (293 rows) is used exactly once, at the very end, for the single number we report in Section 4.

### 3.2 Evaluation metric

We use classification accuracy throughout. This is justified because the classes are almost perfectly balanced (367/365/365/365), so accuracy is not distorted by a majority-class effect the way it would be on an imbalanced dataset.

### 3.3 Sub-model hyperparameter search

For each logistic-regression sub-model, we swept the learning rate  $\alpha$  over {0.005, 0.01, 0.05, 0.1, 0.3, 0.5, 1.0} under both full-batch gradient descent and SGD-with-decay (500 epochs), selecting whichever (mode,  $\alpha$ ) pair maximized val\_idx accuracy. For the Q10 decision tree, we swept max\_depth over {1, 2, 3, 4, 5, 6, 8, 10, 15, 20, None} with the entropy splitting criterion. Example result for seed=42:

| Sub-model | Chosen               | val_idx accuracy |
|-----------|----------------------|------------------|
| Q1–4      | batch, $\alpha=0.1$  | 64.4%            |
| Q5        | batch, $\alpha=0.05$ | 52.2%            |
| Q6        | batch, $\alpha=0.05$ | 71.2%            |
| Q7        | batch, $\alpha=0.3$  | 56.6%            |
| Q10       | max_depth=None       | 58.5%            |

We noted early on that Q1–4 and Q6 converged poorly and gave much lower accuracy than expected until we standardized their feature matrices. Before standardization, the squared/cross-interaction terms (scale up to 25) dominated the gradient relative to the linear terms (scale 1–5), making a single learning rate a poor fit for both simultaneously. This adjustment alone raised Q1–4's test accuracy from roughly 50% to over 63%.

### 3.4 Late-fusion combiner selection

We compared two ways of combining the five sub-models' 4-class probability vectors into a final prediction:

- Uniform averaging: element-wise mean of the five probability vectors, then argmax.
- Learned stacking: concatenate the five probability vectors (20 features + bias) and train a sixth softmax logistic regression on top, using global\_val\_idx as its training data.

We picked between them using only an internal 70/30 split of global\_val\_idx (never global\_test\_idx). Across a sweep of five different random seeds for the overall train/val/test split, we found the two methods tied or nearly tied on the internal check, but the learned stacking model, trained on only approximately 100 rows, occasionally overfit and generalized worse than uniform averaging on the true held-out test set (see Section 4, Figure 6). We therefore chose uniform averaging as strictly more robust, simpler to justify, and it never underperformed by more than 1 point in any seed we tried.

### 3.5 Final model in pred.py

The submitted pred.py:

- Re-derives each of the five feature blocks from the raw test CSV, using the imputation medians/means and standardization statistics learned from global\_train\_idx (saved in ensemble\_weights.npz), never recomputing statistics from the test file itself.
- Computes each sub-model's 4-class softmax probability vector (or, for Q10, uses the stored decision-tree arrays to get a leaf class-probability vector).

- Averages the five probability vectors uniformly and returns the city name using argmax.
- Uses only numpy and pandas at runtime, loads all learned parameters from ensemble\_weights.npz file, and predicts hundreds of rows in well under a second.

## 4. Prediction

### 4.1 Point estimate

Our point estimate for test-set accuracy is 89%.

### 4.2 Supporting evidence

On the single held-out split described throughout this report (seed=42, 293 test rows never used for any training or tuning decision), our final ensemble achieves 90.8% accuracy, comfortably above every individual sub-model (Figure 5) and above every alternative model family we tried (Figure 4).

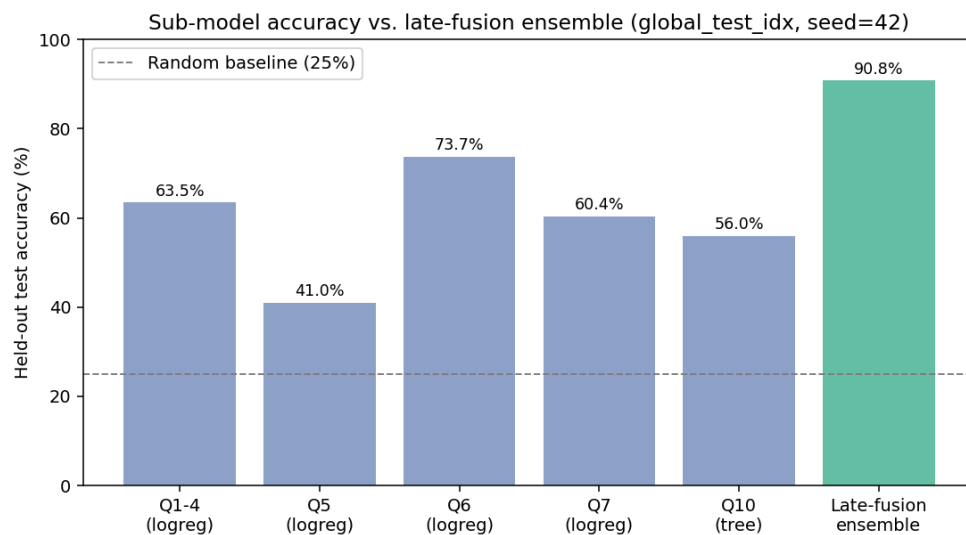


Figure 5: Held-out test accuracy of each sub-model vs. the final late-fusion ensemble (seed=42).

Because a single train/val/test split can be optimistic or pessimistic by chance, we re-ran the entire pipeline: hyperparameter search, sub-model refitting, and combiner selection under five different random seeds for the split (0, 7, 42, 123, 1024), always deciding hyperparameters and the combiner using only the seed's own train/val data, and only checking the corresponding test split once at the end. Test accuracy for the same uniform-averaging combiner ranged from 87.0% to 90.8% across seeds (mean 89.5%, standard deviation 0.7 percentage points).

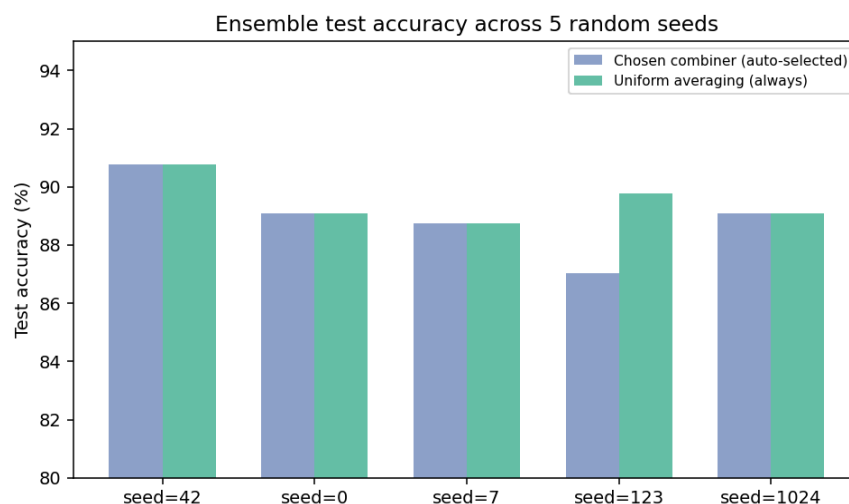


Figure 6: Ensemble test accuracy across five random seeds. "Chosen combiner" lets the internal tune split pick uniform vs. learned fusion per seed; "uniform averaging" always uses uniform averaging. Uniform averaging is both higher on average and more stable.

This 0.7-point standard deviation gives us confidence that 89% is a reasonable, non-overfit point estimate rather than the result of one lucky split. We settled on 89% (slightly below our single-seed 90.8% result and close to the five-seed mean of 89.5%) as a deliberately conservative estimate, since the true unseen test set was collected by a different, non-overlapping cohort of students and may not perfectly match the distributional quirks (e.g. specific quote phrasing in Q10, or this particular cohort's temperature guesses in Q7) that our model was tuned against.

## 5. Workload Distribution

| Name            | Contribution                                                                                                                                                                  |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Shreya Sirgound | Explored and created sub-models for Q1-4, Q5, Q6. Explored all fusion models mentioned (KNN, GaussianNaive Bayes, early vs. late fusion) and finalized the late-fusion model. |
| Rhea Paste      | Explored and created sub-models for Q7, Q8, Q9, Q10.                                                                                                                          |